

Resumo do algoritmo implementado

O algoritmo implementado é a **exponenciação rápida recursiva** (também chamada de *binary exponentiation*).

Ideia principal:

- Quando `exp == 0`, retorna `1` (caso base).
- Calcula recursivamente a potência para `exp // 2`.
- Eleva esse resultado ao quadrado.
- Se `exp` for ímpar, multiplica mais uma vez por `base`.

Com isso, a cada chamada recursiva o expoente é reduzido pela metade, o que torna o método muito mais eficiente que a multiplicação repetida ingênua.

Célula de código 1: função principal `pow_rapido`

Esta célula implementa diretamente o algoritmo do enunciado em Python, preservando a mesma lógica recursiva.

```
In [2]: def pow_rapido(base: int, exp: int) -> int:
        """Calcula base**exp por exponenciação rápida recursiva."""
        if exp == 0:
            return 1

        ret = pow_rapido(base, exp // 2)
        ret = ret * ret

        if exp % 2 == 1:
            ret = ret * base

        return ret
```

Célula de código 2: função instrumentada para análise

Esta versão retorna também o número de chamadas recursivas e a profundidade máxima da recursão para observar o comportamento assintótico.

```
In [3]: def pow_rapido_com_metricas(base: int, exp: int, nivel: int = 1):
        """
        Retorna uma tupla:
        (resultado, numero_de_chamadas, profundidade_maxima)
        """
        if exp == 0:
            return 1, 1, nivel

        sub_resultado, chamadas_sub, profundidade_sub = pow_rapido_com_metricas(base, exp // 2, nivel + 1)

        resultado = sub_resultado * sub_resultado
        if exp % 2 == 1:
            resultado *= base

        chamadas_totais = 1 + chamadas_sub
        profundidade_maxima = max(nivel, profundidade_sub)

        return resultado, chamadas_totais, profundidade_maxima
```

Célula de código 3: exemplos de execução

Aqui validamos a função e exibimos métricas para alguns valores de expoente.

```
In [4]: casos_teste = [0, 1, 2, 5, 10, 31]
        base = 2

        for exp in casos_teste:
            valor = pow_rapido(base, exp)
            valor_m, chamadas, profundidade = pow_rapido_com_metricas(base, exp)

            print(
                f"base={base}, exp={exp:>2} -> resultado={valor}, "
                f"chamadas={chamadas}, profundidade={profundidade}"
            )

            assert valor == valor_m
            assert valor == base ** exp
```

base=2, exp= 0 -> resultado=1, chamadas=1, profundidade=1
base=2, exp= 1 -> resultado=2, chamadas=2, profundidade=2
base=2, exp= 2 -> resultado=4, chamadas=3, profundidade=3
base=2, exp= 5 -> resultado=32, chamadas=4, profundidade=4
base=2, exp=10 -> resultado=1024, chamadas=5, profundidade=5
base=2, exp=31 -> resultado=2147483648, chamadas=6, profundidade=6

Resultados pedidos no enunciado

Considere $n = exp$.

Relação de recorrência (tempo):

- Caso base: $T(0) = \Theta(1)$
- Caso geral: $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$, para $n > 0$

Solução da recorrência:

- Como o problema reduz para metade a cada chamada, temos aproximadamente $k \approx \log_2(n)$ níveis.
- Logo, $T(n) = \Theta(\log n)$.

Complexidade de tempo:

- Melhor caso: $\Theta(1)$ quando $exp = 0$.
- Pior caso: $\Theta(\log n)$ para $n > 0$.

Complexidade de espaço (pilha de recursão):

- Melhor caso: $\Theta(1)$ quando $exp = 0$.
- Pior caso: $\Theta(\log n)$, pois a profundidade da recursão é proporcional a $\log n$.